

Practical:6

Implementation of chain matrix multiplication using dynamic programming.

/08/2026

Aim:

Implementation of chain matrix multiplication using dynamic programming.

Algorithm:

Algorithm:-

input:- Number of matrices "n"
 * dimensions of each matrix in the form of an array P[]
 * if there are n matrices P[] contains n+1 dimensioned

Output:- Display the minimum of scalar multiplication required

Step1:- start

Step2:- Read the number of matrices "n"

Step3:- store dimensions of matrices in array P[]

Step4:- create a table m[i][j] where:
 $m[i][j] = \text{minimum cost of multiply } A_i \text{ through } A_j$

Step5:- consider chains of length 2, 3, ... n

Step6:- for all chains $A_i \dots A_j$ for each function $(A_i \dots A_k) \times (A_{k+1} \dots A_j)$
 calculate cost = $m[i][k] + m[k+1][j] + P[i-1] * P[k] * P[j]$

Step7:- Store minimum cost in m[i][j]

Step8:- finally m[i][j] gives the minimum no. of multiplication.

Program:

```
#include <iostream>
```

```
#include <climits>
```

```
using namespace std;
```

```
void printTable(int m[][100], int n) {  
    cout << "\nDP Table (Minimum Costs):\n";
```



```
for (int i = 1; i < n; i++) {
    for (int j = 1; j < n; j++) {
        if (i > j)
            cout << " - ";
        else
            cout << m[i][j] << "\t";
    }
    cout << endl;
}

int matrixChainOrder(int p[], int n) {
    int m[100][100];

    // Cost of multiplying one matrix is 0
    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    // L = chain length
    for (int L = 2; L < n; L++) {

        for (int i = 1; i < n - L + 1; i++) {

            int j = i + L - 1;

            m[i][j] = INT_MAX;

            // Try every possible split
            for (int k = i; k < j; k++) {

                int q = m[i][k]
                    + m[k + 1][j]
                    + p[i - 1] * p[k] * p[j];

                if (q < m[i][j])
```

```
        m[i][j] = q;
    }
}
}

printTable(m, n);

return m[1][n - 1];
}

int main() {

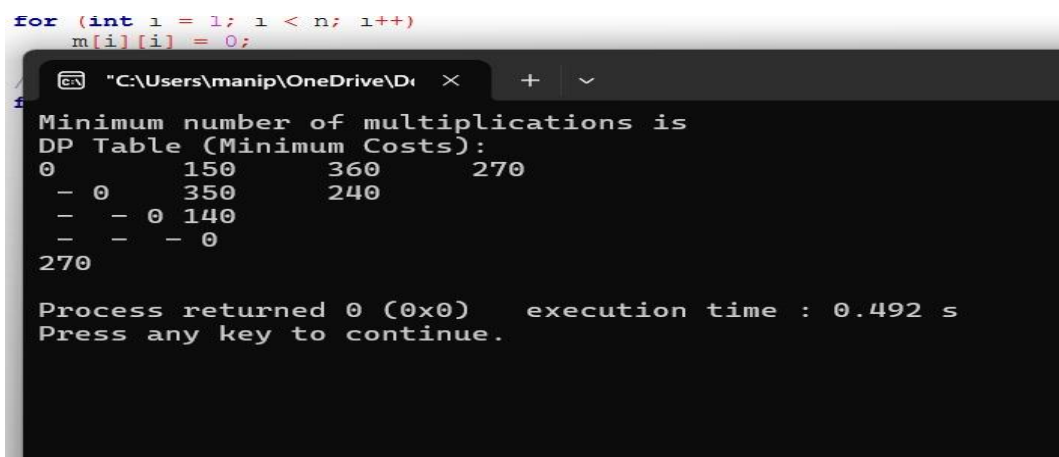
    int arr[] = {3, 5, 10, 7, 2};

    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Minimum number of multiplications is "
        << matrixChainOrder(arr, n) << endl;

    return 0;
}
```

Output:

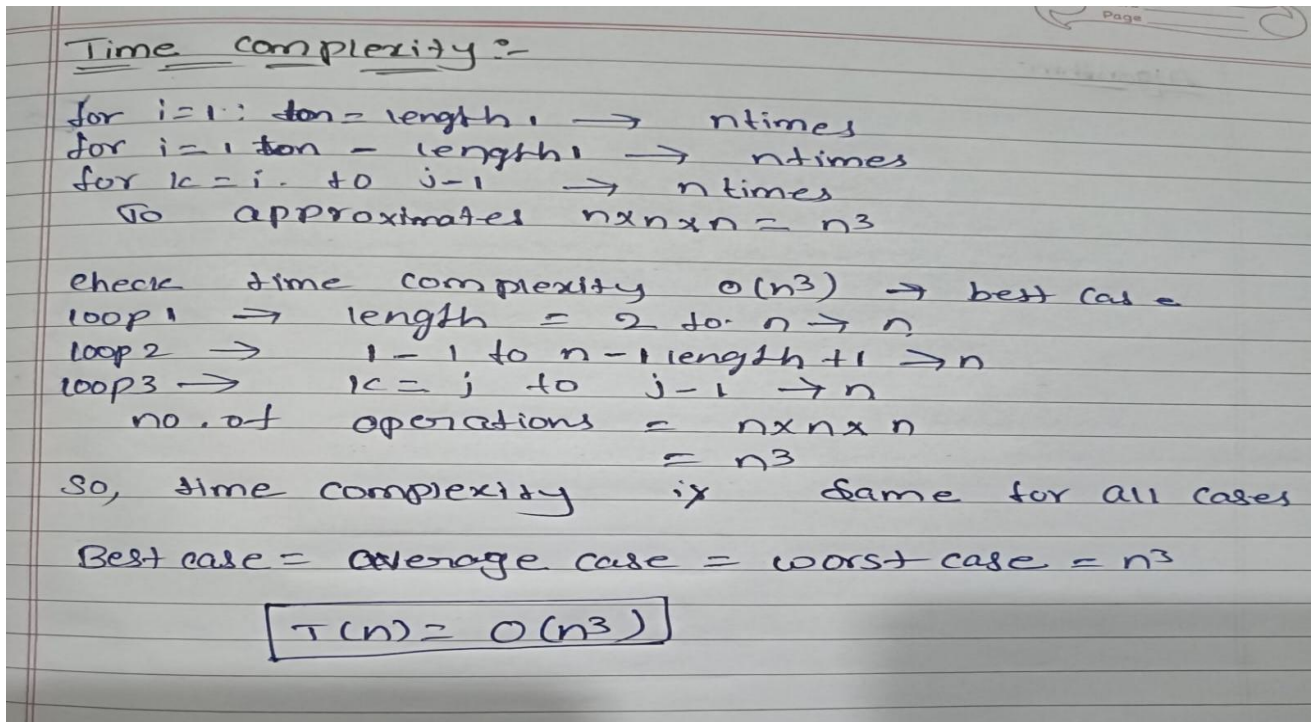


```
for (int i = 1; i < n; i++)
    m[i][i] = 0;

Minimum number of multiplications is
DP Table (Minimum Costs):
0      150      360      270
- 0      350      240
- - 0 140
- - - 0
270

Process returned 0 (0x0)    execution time : 0.492 s
Press any key to continue.
```

Time complexity:



Conclusion:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The algorithm finds the most efficient order of multiplying matrices by minimizing the total number of scalar multiplications. It avoids repeated calculations and provides an optimal solution efficiently.